Relatório da Etapa 2 — Operação, interface e desempenho

Projeto Final de Laboratório de Banco de Dados — UCB 2026/2 — Prof. Samuel Novais
Tema: Escola de Música · **SGBD:** MySQL 8 · **Aplicação:** Python 3 + Flask (web)

Integrantes: Gabriel Gomes Giani, Paulo Henrique Barreto da Rocha Dantas, Letícia Paolino e Lucas de Oliveira Faustino.

Este relatório cobre os itens B1 a B8 da Etapa 2. No arquivo `docs/relatorio-final.pdf` ele vem depois do relatório completo da Etapa 1 (A1 a A5), formando o relatório consolidado pedido no item B8. As alterações feitas no modelo desde a Etapa 1 estão na seção 9.

---

1. Como reproduzir tudo do zero

Passo	Comando	O que faz
1	`sql/01_ddl.sql`	Cria o banco e as 15 tabelas da Etapa 1
2	`sql/02_carga.sql`	Carga de dados fictícios
3	`sql/03_consultas.sql`	As 15 consultas da Etapa 1
4	`sql/04_seguranca.sql`	USUARIO_APP, visões e papéis (B5)
5	`sql/05_auditoria.sql`	Tabela AUDITORIA e gatilhos (B6)
6	`sql/06_otimizacao.sql`	EXPLAIN antes/depois e índices (B4)
7	`sql/07_transacoes.sql` (com `--force`)	COMMIT, ROLLBACK e roteiro de concorrência (B3)
8	`python ferramentas/criar_usuarios_banco.py`	Cria os usuários do MySQL, com as senhas do `.env`
9	`python app/app.py`	Sobe a aplicação em http://127.0.0.1:5000

O passo a passo detalhado, inclusive para o laboratório da faculdade, está no `README.md`. As saídas reais de cada script, geradas na execução que fundamenta este relatório, estão em `docs/resultados-*.txt`.

---

2. B1 — Aplicação com interface

A aplicação é web, feita em Flask, com páginas HTML simples. Toda a lógica de banco fica em `app/repositorio.py`.

Exigência	Onde está	Tela
CRUD completo de 4 entidades	ALUNO (PESSOA + ALUNO), TURMA, INSTRUMENTO e SALA	Alunos, Turmas, Instrumentos, Salas
2 consultas parametrizadas com junção e agregação	Desempenho por instrumento; Inadimplência no período	Consulta: desempenho / inadimplência
Relatório alimentado por visão	`vw_relatorio_financeiro` e `vw_ocupacao_turma`	Relatório
Operação que altera 2+ tabelas em uma transação	Matrícula: MATRICULA + HISTORICO_SITUACAO + MENSALIDADE	Matrícula
Erro de restrição tratado	`app/erros.py`	Qualquer tela

As duas consultas parametrizadas

1. **Desempenho por instrumento.** Filtros: instrumento e média mínima. A consulta junta AVALIACAO, MATRICULA, `vw_aluno_contato`, TURMA e NIVEL, agrupa por aluno e turma e filtra com `HAVING AVG(nota) >= ?`. O nome do aluno vem da visão, e não da tabela PESSOA, por isso a mesma tela funciona também para o perfil PROFESSOR, que não tem acesso a PESSOA.
2. **Inadimplência no período.** Filtros: vencimento de/até e total devido mínimo. A consulta junta MENSALIDADE, MATRICULA e PESSOA, filtra as mensalidades em aberto e já vencidas, agrupa por aluno e filtra com `HAVING SUM(valor) >= ?`.

Tratamento de erros. O usuário nunca vê a exceção do driver. `app/erros.py` lê o **nome da restrição** violada e devolve a mensagem da regra de negócio. Por isso as restrições foram nomeadas na Etapa 1. Exemplos testados na aplicação:

Ação	Erro do MySQL	Mensagem exibida
Cadastrar CPF repetido	1062 `uq_pessoa_cpf`	Já existe uma pessoa cadastrada com este CPF (RN01).
CPF com 3 dígitos	3819 `ck_pessoa_cpf`	O CPF deve ter exatamente 11 dígitos, só números (RN01).
Turma com 50 vagas	3819 `ck_turma_vagas`	A turma deve ter de 1 a 20 vagas (RN13).
Excluir aluno com matrícula	1451 `fk_matricula_aluno`	Não é possível excluir: o aluno ou a turma tem matrículas (RN16).
Matricular duas vezes	1062 `uq_matricula_aluno_turma`	Este aluno já está matriculado nesta turma (RN14).
Secretaria cadastra instrumento	1142 (sem permissão)	Seu perfil não tem permissão para esta operação (o banco recusou).

Duas regras que o MySQL não consegue expressar em CHECK são verificadas pela aplicação antes de gravar. A **RN03** exige que a data de nascimento não seja futura, e o MySQL não aceita `CURDATE()` dentro de CHECK. A **RN04** exige responsável para menor de idade.

---

3. B2 — Camada de acesso a dados

Consultas parametrizadas. Todo valor digitado pelo usuário entra no SQL como parâmetro (`%s`), e o driver PyMySQL o envia separado do comando. A aplicação não monta nenhum SQL concatenando texto do usuário. Nomes de tabela e coluna não podem ser parâmetro. Nos poucos pontos em que eles variam, no CRUD genérico, os nomes vêm de uma lista fixa definida no código (`ENTIDADES`), nunca da requisição.

Demonstração de tentativa de injeção de SQL (tela Alunos, campo de busca):

- Entrada digitada: `' OR 1=1 --`
- O que a aplicação executa: `... WHERE p.nome LIKE %s`, com o parâmetro `"' OR 1=1 -- %"`. O banco procura um nome que **começa com esse texto**.
- **Resultado obtido: 0 linhas.**
- Para comparar: se o SQL fosse montado por concatenação, o comando viraria `WHERE p.nome LIKE '' OR 1=1 -- %'`. O `OR 1=1` vale para todas as linhas e o `--` comenta o resto. Executado diretamente no MySQL, esse comando devolveu **os 47 alunos**.

Credenciais fora do código. As senhas do banco e a chave da sessão ficam no arquivo `.env`, que está no `.gitignore`. O repositório traz só o modelo, `.env.example`. Os usuários do MySQL também não são criados por script SQL, porque a senha ficaria versionada. Eles são criados por `ferramentas/criar_usuarios_banco.py`, que lê as senhas do `.env`.

---

4. B3 — Transações e controle de concorrência

4.1 A operação transacional

Matricular aluno (`repositorio.matricular()` na aplicação e `sql/07_transacoes.sql` no terminal):

```
START TRANSACTION
  SELECT vagas, id_nivel, data_fim FROM turma WHERE id_turma = ? FOR UPDATE   -- trava a turma
  SELECT COUNT(*) FROM matricula WHERE id_turma = ? AND situacao = 'ATIVA'    -- RN13 vagas
  (verifica pré-requisito concluído - RN28)
  INSERT INTO matricula ...
  INSERT INTO historico_situacao ...                                         -- RN17
  INSERT INTO mensalidade ...
COMMIT        (ou ROLLBACK em qualquer erro)
```

As três escritas precisam acontecer juntas. Matrícula sem mensalidade é aluno estudando sem cobrança. Mensalidade sem matrícula é cobrança de quem não estuda.

4.2 Falha provocada no meio da transação

O terceiro INSERT usa valor 0 e viola `ck_mensalidade_valor` (RN24). Resultado real (`docs/resultados-transacoes.txt`):

Momento	Matrículas do aluno 24 na turma 8
Dentro da transação, depois do erro	**1**
Depois do ROLLBACK	**0**
Históricos órfãos no banco	**0**

Ponto importante observado: no MySQL, um erro desfaz **só o comando que falhou**, não a transação inteira. A matrícula inserida antes continuava existindo dentro da sessão. Quem a desfaz é o `ROLLBACK` explícito, que na aplicação fica no bloco `except`. Sem ele, o próximo `COMMIT` gravaria uma matrícula sem mensalidade. Na aplicação, o mesmo teste (valor 0 na tela Matrícula) mostrou a mensagem de erro e não deixou nenhum registro.

4.3 Experimento com duas sessões simultâneas

Turma 7: 8 vagas, 7 ocupadas, sobra **uma**. Dois alunos tentam a última vaga ao mesmo tempo. O experimento foi executado com duas conexões reais em paralelo (`ferramentas/experimento_concorrencia.py`). Saída completa em `docs/resultados-concorrencia.txt`.

Nível de isolamento em vigor: REPEATABLE READ (padrão do InnoDB).

Caso	Comportamento observado	Resultado
Sem trava (só `SELECT COUNT`)	As duas sessões leram 7 ocupadas. Nenhuma esperou, as duas inseriram e fizeram COMMIT.	**9 matrículas numa turma de 8 vagas**
Com trava (`SELECT ... FOR UPDATE` na turma)	A sessão B **ficou bloqueada 1,48 s**, até o COMMIT da A. Depois viu 8 ocupadas e fez ROLLBACK.	**8 matrículas, correto**

Se a sessão A demorar mais que `innodb_lock_wait_timeout` (50 s), a B recebe o erro 1205, que a aplicação traduz como "Outra operação está usando este registro".

Justificativa do nível de isolamento. Mantivemos REPEATABLE READ, por três motivos:

- O problema do caso "sem trava" não é o nível de isolamento. É o padrão "ler, decidir e gravar" sem trava. READ COMMITTED teria o mesmo defeito.
- SERIALIZABLE evitaria o defeito sozinho, mas transforma toda leitura em leitura com trava compartilhada. As telas de consulta e o relatório passariam a bloquear e a ser bloqueados pelas matrículas.
- Com REPEATABLE READ e `FOR UPDATE` só na linha da turma, a trava fica exatamente onde existe conflito: duas matrículas na **mesma** turma. Matrículas em turmas diferentes continuam em paralelo.

Detalhe de implementação: o `FOR UPDATE` é a **primeira** leitura da transação. No REPEATABLE READ, a "foto" das leituras comuns é tirada na primeira leitura comum. Travando antes, a

contagem de vagas já enxerga o que a outra sessão gravou.

---

5. B4 — Otimização de consultas

Volume: com a carga da Etapa 1 (54 pessoas, 246 mensalidades), toda consulta roda abaixo de 1 ms e índice nenhum faz diferença mensurável. Por isso `06_otimizacao.sql` gera volume sintético (**50.054 pessoas** e **30.918 mensalidades**), mede, e no fim apaga os dados sintéticos. Os índices ficam. Os tempos abaixo são o `actual time` do `EXPLAIN ANALYZE` no MySQL 8.0. A saída completa está em `docs/resultados-otimizacao.txt`.

Consulta	Plano antes	Índice criado	Plano depois	Tempo antes	Tempo depois
1. Busca por início do nome	Table scan (50.054 linhas) + sort	`idx_pessoa_nome (nome)`	Covering index range scan (416 linhas)	12,3 ms	**0,18 ms**
2. Mensalidades vencidas e não pagas	Index lookup em `data_pagamento` (4.394 linhas) + filtro	`idx_mensalidade_aberto_venc (data_pagamento, data_vencimento)`	Index range scan (1.217 linhas)	6,8 ms	**1,4 ms**
3. Relatório financeiro por mês	Table scan + tabela temporária	`idx_mensalidade_comp_cobertura (competencia, data_pagamento, valor)`	Covering index scan + group aggregate	16,6 ms	**8,2 ms**

Leitura dos planos e escolha das colunas

Consulta 1. Antes, `type = ALL`: o banco leu as 50 mil linhas, testou o `LIKE` em cada uma e ordenou o resultado. `LIKE 'texto%'`, com o curinga só no final, é uma faixa ordenada da árvore B, então o índice em `nome` serve para filtrar **e** para o `ORDER BY`. Como o InnoDB guarda a chave primária em todo índice secundário, `(nome)` já contém `id_pessoa`, e a consulta é respondida só pelo índice ("covering"). Observação: `LIKE '%texto'` não aproveitaria índice nenhum.

Consulta 2. Já existia um índice só em `data_pagamento`, da Etapa 1. Ele achava as 4.394 mensalidades em aberto, mas o banco ainda lia cada uma para testar o vencimento, e 72% eram descartadas. No índice composto, a **ordem importa**: `data_pagamento` vem primeiro porque o teste é de **igualdade** (IS NULL), e `data_vencimento` depois porque é **faixa** (<). Depois de uma coluna de faixa, as colunas seguintes do índice não filtram mais. O índice antigo virou prefixo do novo e foi removido, porque seria só custo de escrita.

Consulta 3 — resultado honesto. É o ganho mais fraco e o menos confiável. A consulta continua lendo **todas** as 30.918 linhas, porque é um relatório do histórico inteiro, e nenhum índice evita isso. O ganho veio de dois detalhes: ler um índice mais estreito que a tabela e agrupar na ordem do índice, sem tabela temporária. Em duas execuções completas do script, o ganho foi de **21%** (17,7 → 14,0 ms) e de **51%** (16,6 → 8,2 ms, a execução salva). Medições de poucos milissegundos variam com cache e carga da máquina. Por isso a conclusão segura é "melhora moderada", não um número exato. Nas consultas 1 e 2 a melhora se repetiu nas duas execuções, porque o número de linhas lidas caiu de verdade (de 50.054 para 416, e de 4.394 para 1.217). Para reduzir de verdade seria preciso ler menos linhas, por exemplo com uma tabela de resumo mensal mantida por gatilho. Nesta escala isso não se justifica, porque 14 ms com 30 mil linhas equivalem a muitos anos de operação da escola.

---

6. B5 — Segurança e controle de acesso

6.1 Perfis e privilégios (`sql/04_seguranca.sql`)

Cada perfil da aplicação conecta ao MySQL com **um usuário de banco diferente**. Quem garante as permissões é o próprio banco, não o menu. Mesmo que alguém burle a tela, o MySQL recusa.

Papel MySQL	Usuário	Pode	Não pode
r_login	app_login	Ler USUARIO_APP e atualizar ultimo_login	Qualquer outra tabela
r_admin	app_admin	SELECT/INSERT/UPDATE/DELETE em tudo	Criar/alterar estrutura (DDL) e usuários
r_secretaria	app_secretaria	Cadastro de alunos e turmas, matrícula, lançar e dar baixa em mensalidade	Instrumentos/salas (só leitura), **apagar mensalidade**, ler AUDITORIA
r_professor	app_professor	Ler turmas; lançar aulas, frequência e notas	Ler PESSOA (CPF, endereço), MENSALIDADE, AUDITORIA

O **REVOKE** demonstrado: a secretaria recebeu DELETE em MENSALIDADE junto com os outros privilégios, e o script o retira com REVOKE DELETE ON mensalidade FROM r_secretaria (RN33: só o administrador exclui mensalidade). Testes com login real:

```
app_professor> SELECT cpf FROM pessoa;              -> ERROR 1142 SELECT command denied
app_professor> SELECT * FROM vw_aluno_contato;      -> funciona (nome, e-mail, telefone, sem CPF)
app_secretaria> DELETE FROM mensalidade WHERE ...;  -> ERROR 1142 DELETE command denied
app_secretaria> SELECT * FROM auditoria;            -> ERROR 1142 SELECT command denied
```

O **mesmo login sob dois perfis** aparece na demonstração: a tela "Consulta: desempenho" funciona para SECRETARIA e PROFESSOR, mas a tela Alunos, com CPF, só funciona para a secretaria. Para o professor o banco recusa e a aplicação mostra a mensagem tratada.

6.2 Visão de restrição

vw_aluno_contato restringe **colunas**: expõe nome, e-mail, telefone, se é menor de idade e o responsável. Esconde CPF, endereço e data de nascimento completa. O professor recebe SELECT na visão e não na tabela PESSOA.

6.3 Senhas com hash e sal

Tabela USUARIO_APP: a senha nunca é gravada. Grava-se o **PBKDF2-HMAC-SHA256 com 200.000 iterações** sobre a senha, com um **sal aleatório de 16 bytes por usuário**.

- O sal faz duas senhas iguais gerarem hashes diferentes e inutiliza tabelas pré-calculadas.
- As 200 mil iterações deixam cada tentativa de força bruta lenta.
- A comparação usa hmac.compare_digest, em tempo constante.
- Login inexistente e senha errada recebem a **mesma** mensagem, para não revelar quais logins existem.

6.4 Dados pessoais no projeto

Dado	Onde	Por que é sensível	Cuidado adotado
CPF	PESSOA	Identifica a pessoa	Só ADMIN e SECRETARIA leem. Professor vê pela visão, sem CPF. Não é chave primária (Etapa 1), então não se espalha pelas tabelas filhas.
Nome, nascimento, e-mail, endereço, telefone	PESSOA, TELEFONE	Dados pessoais (LGPD)	Mesmo controle do CPF. Endereço e nascimento ficam fora da visão do professor.
Responsável e dados de menor de idade	ALUNO	Dados de criança e adolescente pedem cuidado reforçado	Acesso restrito como acima. O professor vê só o indicador "menor de idade" e o nome do responsável, necessário para contato.

Dado	Onde	Por que é sensível	Cuidado adotado
Situação financeira	MENSALIDADE	Expõe a vida financeira do aluno	Professor não tem acesso. Alterações auditadas. Secretaria não apaga.
Senhas	USUARIO_APP	Credencial	Só hash + sal.
Credenciais do banco	`.env`	Acesso total aos dados	Fora do repositório.

Toda a carga é **fictícia**, gerada por `ferramentas/gerar_carga.py`. Nenhum dado real de terceiros foi usado.

---

7. B6 — Gatilhos e auditoria

`sql/05_auditoria.sql` cria a tabela AUDITORIA e dois gatilhos, AFTER UPDATE. Cada registro guarda **autor** (login da aplicação e usuário do MySQL), **data e hora**, e os **valores anterior e posterior** em JSON.

Gatilho	Tabela	Por que auditar (pelo domínio)
`trg_mensalidade_au`	MENSALIDADE	É dinheiro. Uma baixa lançada por engano ou um valor alterado vira prejuízo para a escola ou cobrança indevida ao aluno. (RN34)
`trg_matricula_au`	MATRICULA	Trancar ou cancelar libera vaga, interrompe cobrança e muda o histórico acadêmico. HISTORICO_SITUACAO diz o que mudou; a auditoria diz quem mudou. (RN35)

Por que não auditamos AULA e FREQUENCIA: são as tabelas de maior movimento e as alterações nelas não têm impacto financeiro. Auditar tudo gera volume sem uso.

Como o autor é registrado. A aplicação executa `SET @usuario_app = '<login>'` ao conectar, e o gatilho grava esse valor e o `USER()` do MySQL. Os gatilhos executam com os privilégios de quem os criou, então secretaria e professor não precisam de permissão na AUDITORIA e não podem lê-la nem apagá-la. Só o ADMIN vê a tela Auditoria.

Exemplo real, uma baixa de pagamento pela tela Mensalidades com o login `secretaria`:

```
tabela: mensalidade   chave: 104   usuario_app: secretaria   usuario_banco: app_secretaria@localhost
antes:  {"valor": 280.00, "data_pagamento": null,         "data_vencimento": "2026-02-10", "forma_pagamento": null}
depois: {"valor": 280.00, "data_pagamento": "2026-10-26", "data_vencimento": "2026-02-10", "forma_pagamento": "PIX"}
```

---

8. B7 — Projeto físico e armazenamento

8.1 Revisão dos tipos de dados

Coluna	Tipo atual	Avaliação
`valor` (mensalidade), `valor_hora`	DECIMAL(8,2) e DECIMAL(7,2)	**Mantém.** Dinheiro nunca em FLOAT, que arredonda (0,1 + 0,2 ≠ 0,3).
`cpf`, `cep`, `uf`	CHAR(11), CHAR(8), CHAR(2)	**Mantém.** Tamanho fixo, e CHAR evita o byte de comprimento.
`situacao`, `tipo`, `familia`, `status`	VARCHAR + CHECK	**Mantém.** ENUM ocuparia 1 byte, mas incluir um valor novo exige ALTER TABLE e reordenar o ENUM muda o significado dos dados. Com CHECK, a mudança é só na restrição.
`id_sala` (TINYINT), `id_instrumento`/`id_nivel` (SMALLINT)		**Mantém.** Uma escola não terá mais que 255 salas nem 65 mil níveis. Chave menor deixa todo índice que a referencia menor.

Coluna	Tipo atual	Avaliação
`telefone.numero`	VARCHAR(15)	**Mudaria para CHAR(11) ou VARCHAR(11).** O CHECK já limita a 10–11 dígitos, então 15 nunca é usado.
`aula.numero_aula`	TINYINT UNSIGNED	**Mantém, com atenção.** Limite de 255 aulas por turma. Cursos longos (anuais com 3 aulas por semana) chegariam perto.
`auditoria.id_auditoria`	BIGINT	**Novo nesta etapa.** Tabela que só cresce e nunca é apagada.
`historico_situacao.data_alteracao`	DATETIME	**Mantém.** DATETIME não sofre com fuso horário como TIMESTAMP e vai além de 2038.

Índices revisados nesta etapa: entraram `idx_pessoa_nome`, `idx_mensalidade_aberto_venc` e `idx_mensalidade_comp_cobertura` (B4). O `idx_mensalidade_pagamento`, da Etapa 1, saiu por ser redundante.

8.2 Estimativa de crescimento (2 anos)

Base: a carga atual tem cerca de 37 matrículas por semestre (74 em dois semestres), com cerca de 13 aulas registradas por turma. Hipótese: crescimento de **20% de matrículas por semestre**, de 37 para 44, 53, 64 e 77, um total de **cerca de 240 matrículas novas em 2 anos**. Por matrícula e semestre: 18 aulas com frequência, 5 mensalidades, 2 avaliações, 2 eventos de histórico e cerca de 5 registros de auditoria.

Tabela	Linhas hoje	Linhas novas em 2 anos	Bytes/linha (dado + índices)	Tamanho em 2 anos
FREQUENCIA (maior movimento)	901	~4.300	~60	< 1 MB
MENSALIDADE	246	~1.200	~120 (3 índices)	< 1 MB
AUDITORIA	0	~1.200	~400 (JSON)	< 1 MB
AVALIACAO, HISTORICO_SITUACAO	127 / 111	~500 cada	~100	< 1 MB

Conclusão: em 2 anos o banco inteiro fica na casa de **poucos megabytes**. O tamanho que o MySQL mostra hoje, de 16 a 48 KB por tabela, é quase só a página mínima de 16 KB do InnoDB, não dado.

8.3 Particionamento e arquivamento

Nenhuma tabela justifica particionamento. Particionar compensa quando uma tabela tem dezenas de milhões de linhas, ou quando se apaga um período inteiro de uma vez. Aqui a maior tabela teria cerca de 5 mil linhas em 2 anos. Além disso, no MySQL a chave de partição precisa fazer parte de toda chave primária e única, o que obrigaria a mudar as PKs, e as tabelas com chave estrangeira não podem ser particionadas.

Proposta para o futuro, só se a escola virar uma rede: **arquivar FREQUENCIA de semestres encerrados** em uma tabela `frequencia_arquivo`, uma vez por ano, já que o dado é imutável depois que o semestre fecha.

8.4 Plano de backup

Item	Definição
O que copiar	O banco `escola_musica` inteiro, com gatilhos e visões: `mysqldump --single-transaction --routines --triggers --events escola_musica`. Também os papéis e usuários do MySQL, que ficam fora do banco: reaplicar `04_seguranca.sql` e `criar_usuarios_banco.py`. O `.env` vai para um cofre de senhas, **não** junto do backup.

Item	Definição
Frequência	Backup **completo diário** às 23h, depois do expediente (pela RN11 a última aula começa às 21h). **Log binário** (binlog) ligado, para recuperar até o minuto anterior a uma falha (point-in-time recovery).
Retenção	Avô-pai-filho: **7 diários**, **4 semanais** (domingo) e **12 mensais** (dia 1). Os mensais cobrem o ano fiscal de mensalidades.
Onde guardar	Uma cópia local e uma em outro lugar (nuvem), criptografada, porque o backup contém dados pessoais.
Teste de restauração	Todo mês: restaurar o backup do dia em um MySQL separado, rodar `03_consultas.sql` e conferir se a consulta C15 (auditoria de regras) retorna 0 em todas as linhas e se a contagem de linhas bate com a produção. O backup só conta como feito se a restauração funcionar.

`--single-transaction` faz o dump ler uma foto consistente sem travar as tabelas. Funciona porque todas são InnoDB, e as matrículas continuam funcionando durante o backup.

---

9. Alterações no modelo desde a Etapa 1

Correções da devolutiva da Etapa 1: não há correções de devolutiva registradas nesta versão. As mudanças feitas no modelo desde a Etapa 1 estão listadas abaixo.

O que mudou no modelo nesta etapa:

Mudança	Motivo
Nova entidade **USUARIO_APP** (login, senha_hash, sal, perfil, id_pessoa)	Login da aplicação com hash e sal (B5)
Nova entidade **AUDITORIA**	Registro de alterações (B6)
Visões `vw_aluno_contato`, `vw_ocupacao_turma`, `vw_relatorio_financeiro`	Restrição de acesso e relatório (B1, B5)
Índices novos e remoção de `idx_mensalidade_pagamento`	Otimização (B4)
`SET NAMES utf8mb4` no início dos scripts	Evita acentos corrompidos ao rodar pelo terminal com codificação diferente de UTF-8
Novas regras **RN31 a RN35**	Ver abaixo

Novas regras de negócio

Regra	Texto	Atendida por
RN31	O login de usuário da aplicação é único.	Restrição `uq_usuario_app_login`
RN32	Todo usuário da aplicação tem exatamente um perfil: ADMIN, SECRETARIA ou PROFESSOR.	Restrição `ck_usuario_app_perfil`
RN33	Somente o administrador exclui mensalidade.	`REVOKE DELETE ... FROM r_secretaria`
RN34	Toda alteração de mensalidade fica registrada com autor, data e valores.	Gatilho `trg_mensalidade_au`
RN35	Toda mudança de situação da matrícula fica registrada com autor, data e valores.	Gatilho `trg_matricula_au`

Dicionário de dados das novas entidades

USUARIO_APP

Atributo	Descrição	Domínio	Obrig.	Observação
id_usuario	Identificador interno	Inteiro sequencial	Sim	Chave primária substituta
login	Nome de acesso	Texto até 30	Sim	RN31 — único
senha_hash	PBKDF2-SHA256 da senha	64 caracteres hexadecimais	Sim	Nunca a senha em texto
sal	Sal aleatório do usuário	32 caracteres hexadecimais	Sim	16 bytes
perfil	Perfil de acesso	ADMIN, SECRETARIA, PROFESSOR	Sim	RN32
id_pessoa	Professor ligado ao login	Inteiro (FK PESSOA)	Não	ON DELETE SET NULL
ativo	Login liberado	Booleano	Sim	Padrão verdadeiro
ultimo_login	Último acesso	Data e hora	Não	

AUDITORIA

Atributo	Descrição	Domínio	Obrig.	Observação
id_auditoria	Identificador	Inteiro sequencial (BIGINT)	Sim	Chave primária
tabela	Tabela alterada	Texto até 30	Sim	
operacao	Tipo de alteração	INSERT, UPDATE, DELETE	Sim	
chave	Chave do registro alterado	Texto até 40	Sim	
valores_antes / valores_depois	Estado anterior e posterior	JSON	Não	RN34, RN35
usuario_app	Login da aplicação	Texto até 30	Não	Nulo se veio do terminal
usuario_banco	Usuário do MySQL	Texto até 100	Sim	USER()
data_hora	Momento da alteração	Data e hora com milissegundos	Sim	Padrão: agora

---

10. Roteiro da demonstração (15 min, item 6.2)

Antes da aula: banco carregado (passos 1 a 8 da seção 1), aplicação rodando e dois terminais MySQL abertos.

Momento	Tempo	O que fazer
Contexto	2 min	Domínio, entidades principais e o que mudou desde a Etapa 1 (seção 9)
CRUD e consultas	4 min	Login `secretaria` → cadastrar aluno → Consulta: inadimplência → Relatório
Transação	3 min	Matrícula com valor 320 (sucesso). Matrícula com valor 0 (falha): mostrar que nada ficou gravado
Restrições em ação	2 min	CPF repetido; excluir aluno com matrícula; turma com 50 vagas
Desempenho e acesso	3 min	`06_otimizacao.sql`: EXPLAIN antes e depois da consulta 1. Login `prof.julia` → tela Alunos recusada pelo banco; desempenho funciona. Nos dois terminais: caso 3b de `07_transacoes.sql`
Arguição	1 min	—

11. Declaração de uso de inteligência artificial (item 9 do enunciado)

A equipe usou o assistente de IA **Claude (Anthropic)**, pelo Claude Code, nesta etapa:

- **Onde foi usado:** redação e revisão dos scripts `04` a `07`, da aplicação Flask (`app/`), das ferramentas `criar_usuarios_banco.py` e `experimento_concorrencia.py` e deste relatório.
- **Como o resultado foi verificado:**
  - Todos os scripts foram executados do início ao fim em um MySQL 8.0 com o banco recriado do zero (`01` → `07`).
  - As saídas reais foram salvas em `docs/resultados-*.txt`. Os números deste relatório (tempos, contagens, resultados de concorrência) foram copiados dessas execuções, não estimados, exceto a estimativa de crescimento da seção 8.2, que é declaradamente uma hipótese.
  - A aplicação foi testada nos três perfis, com casos de sucesso e de erro (seção 2).
- **Responsabilidade:** a equipe revisou o material e é responsável por explicar qualquer decisão na arguição.